VQE

Genesis Silesky

21 de octubre de 2026

1. VQE: eigensolver cuántico variacional

1.1. Objetivo

Calcular la energía del estado fundamental de un Hamiltoniano con un algoritmo híbrido: un circuito cuántico parametrizado prepara estados de prueba y un optimizador clásico ajusta los parámetros.

1.2. Conceptos previos

Concepto previo: valor esperado, Hamiltoniano y era NISQ

Ver capítulos 08 (era NISQ), 12 (Hamiltoniano) y 13 (valores esperados de Pauli). El *valor esperado* de H en el estado $|\psi\rangle$ es $\langle H\rangle = \langle\psi| H |\psi\rangle = \mathrm{Tr}(H\rho)$ con $\rho = |\psi\rangle\langle\psi|$.

1.3. Fundamento teórico

Los computadores cuánticos actuales tienen ruido y pocos qubits (era NISQ, término de John Preskill de 2018). En ese contexto son muy usados los algoritmos **híbridos cuántico-clásicos**: parte del cálculo corre en un computador clásico y parte en uno cuántico. El VQE es uno de ellos; su tarea es calcular la energía aproximada del estado fundamental de un Hamiltoniano. Se aplica sobre todo en simulación de química cuántica (moléculas como el agua o cadenas de hidrógeno) y también en optimización y aprendizaje automático cuántico.

Principio variacional. Para cualquier estado de prueba $|\psi\rangle$,

$$E_0 \;\leq\; \frac{\langle\psi| H |\psi\rangle}{\langle\psi|\psi\rangle}, \tag{1}$$

donde E_0 es la energía del estado fundamental. Por eso, minimizar el valor esperado acerca el resultado a E_0.

Los pasos del VQE.

1. **Hamiltoniano:** se escribe el problema como un Hamiltoniano de operadores de Pauli.
2. **Estado de prueba:** un circuito parametrizado (rotaciones con parámetro y puertas de dos qubits como CNOT o CZ) se aplica a $|0\ldots0\rangle$.
3. **Valor esperado:** se mide $\langle H\rangle$ en el computador cuántico y se envía al clásico.
4. **Optimización:** un método clásico (con o sin gradiente) actualiza los parámetros.
5. Se repiten los pasos 3 y 4 hasta converger o llegar al máximo de iteraciones.
6. La última iteración da el autoestado y el autovalor óptimos.

Regla del desplazamiento de parámetros. Para obtener el gradiente se evalúa el circuito con el parámetro movido:

$$\frac{dE}{d\theta} = \frac{E(\theta + s) - E(\theta - s)}{2} \quad (s = \pi/2), \qquad \theta \leftarrow \theta - \eta \frac{dE}{d\theta}, \tag{2}$$

con η la tasa de aprendizaje. Esta forma (con $s = \pi/2$) es exacta para puertas de rotación $e^{-i\theta P/2}$; con un desplazamiento δ cualquiera se obtiene solo una aproximación.

Cálculo de $\langle H \rangle$. Se puede hacer con el vector de estado, $\langle\psi| H |\psi\rangle = \sum_{i,j} \bar{\psi}_i H_{ij} \psi_j$, o con la traza, $\mathrm{Tr}(H\rho)$. Dan el mismo resultado.

El caso del ejemplo. El modelo de Heisenberg de dos qubits, $H = XX + YY + ZZ$, tiene energías -3 (estado fundamental, el singlete $(|01\rangle - |10\rangle)/\sqrt{2}$) y $+1$ (tres veces). El circuito de prueba usa dos parámetros: $R_y(\theta_1)$ sobre q_0, una CNOT $q_0 \to q_1$ y $R_x(\theta_2)$ sobre q_0.

1.4. Código (completamente comentado)

```python
# %% [markdown]
# # 19 - VQE: encontrar la energia del estado fundamental con un circuito parametrizado

# %% Celda 1: importaciones
import numpy as np
from qiskit import QuantumCircuit
from qiskit.quantum_info import DensityMatrix, SparsePauliOp, Statevector
import matplotlib.pyplot as plt

# %% Celda 2: el problema
# Se busca la energia mas baja de un Hamiltoniano (la energia del estado fundamental).
# Aqui: el modelo de Heisenberg de dos qubits,  H = XX + YY + ZZ.
H = SparsePauliOp(["XX", "YY", "ZZ"])
H_matriz = H.to_matrix()
# Como el sistema es pequeno, se puede diagonalizar para saber la respuesta exacta.
print("Energias exactas de H:", np.linalg.eigvalsh(H_matriz).round(4))
print("Energia del estado fundamental:", float(np.linalg.eigvalsh(H_matriz).min()))
# VQE busca ese minimo SIN diagonalizar, usando solo valores esperados.

# %% Celda 3: dos formas de calcular el valor esperado <H>
# (1) Con el vector de estado:   <H> = <psi| H |psi>.
# (2) Con la matriz densidad:    <H> = tr(H rho),   rho = |psi><psi|.
# Dan el mismo numero. Se prueba con un estado cualquiera.
psi = Statevector(np.array([1, 1j, -1, 0.5]) / np.linalg.norm([1, 1j, -1, 0.5]))
via_vector = np.real(psi.data.conj() @ H_matriz @ psi.data)
via_traza = np.real(np.trace(H_matriz @ DensityMatrix(psi).data))
print(f"<psi|H|psi> = {via_vector:.4f} | tr(H rho) = {via_traza:.4f}")


# %% Celda 4: el circuito parametrizado ("ansatz")
# El estado de prueba |psi(theta)> sale de aplicar un circuito con parametros
# theta = (theta1, theta2) al estado inicial |00>:
#     Ry(theta1) en q0, CNOT(q0 -> q1), Rx(theta2) en q0.
def ansatz(theta):
    qc = QuantumCircuit(2)
    qc.ry(theta[0], 0)
    qc.cx(0, 1)
    qc.rx(theta[1], 0)
    return qc


def energia(theta):
```

```
    """Valor esperado <H> del estado de prueba (via tr(H rho))."""
    rho = DensityMatrix(Statevector(ansatz(theta)))
    return float(np.real(np.trace(H_matriz @ rho.data)))


# En Colab o Jupyter el circuito se dibuja solo; en terminal no se muestra.
ansatz([0.3, 0.5]).draw()

# %% Celda 5: regla del desplazamiento de parametros (parameter-shift)
# Para saber hacia donde mover cada parametro hace falta el gradiente dE/dtheta.
# Con una puerta de rotacion exp(-i theta P / 2) se puede obtener exacto con
# solo dos evaluaciones del circuito:
#     dE/dtheta = ( E(theta + s) - E(theta - s) ) / 2,    con s = pi/2.
def gradiente(theta, s=np.pi / 2):
    g = np.zeros_like(theta)
    for i in range(len(theta)):
        mas, menos = theta.copy(), theta.copy()
        mas[i] += s
        menos[i] -= s
        g[i] = (energia(mas) - energia(menos)) / 2
    return g


# Se comprueba contra una derivada numerica con un paso muy pequeno.
theta0 = np.array([1.1, 2.3])
eps = 1e-6
numerico = np.array([(energia(theta0 + eps * e) - energia(theta0 - eps * e)) / (2 * eps) for e
    ↪ in np.eye(2)])
print("Gradiente (parameter-shift):", gradiente(theta0).round(5))
print("Gradiente (numerico)       :", numerico.round(5))

# %% Celda 6: el ciclo de VQE
# 1. Elegir parametros iniciales.
# 2. Calcular <H> (computador cuantico).
# 3. Calcular el gradiente con parameter-shift.
# 4. Actualizar:  theta <- theta - lr * gradiente  (computador clasico).
# 5. Repetir hasta que <H> converja.
theta = np.array([1.3, 2.0])      # parametros iniciales (radianes)
lr = 0.3                          # tasa de aprendizaje
historia = [energia(theta)]
print("iteracion | theta1  theta2 |  <H>")
for it in range(1, 41):
    theta = theta - lr * gradiente(theta)
    historia.append(energia(theta))
    if it <= 5 or it % 10 == 0:
        print(f"{it:9d} | {theta[0]:6.3f} {theta[1]:6.3f} | {historia[-1]:+.4f}")
print(f"Energia final = {historia[-1]:.4f} (exacta: -3)")

# %% Celda 7: grafica de convergencia
plt.plot(historia, "o-")
plt.axhline(-3, color="gray", linestyle="--", label="energía exacta (-3)")
plt.xlabel("iteración")
plt.ylabel("<H>")
plt.title("Convergencia de VQE")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

1.5. Explicación celda por celda

Celda 2 Define $H = XX + YY + ZZ$ y calcula sus energías exactas por diagonalización, solo para comparar.

Celda 3 Calcula $\langle H \rangle$ de un estado de prueba de las dos formas y muestra que coinciden.

Celda 4 Define el circuito de prueba y la función de energía $\mathrm{Tr}(H\rho)$.

Celda 5 Implementa la regla del desplazamiento y la compara con una derivada numérica.

Celda 6 El ciclo de VQE: 40 iteraciones con tasa de aprendizaje 0.3.

Celda 7 Gráfica de convergencia.

1.6. Salida obtenida

```
Energias exactas de H: [-3.  1.  1.  1.]
Energia del estado fundamental: -3.0
<psi|H|psi> = -0.2308 | tr(H rho) = -0.2308
Gradiente (parameter-shift): [ 0.75582 -0.08113]
Gradiente (numerico)       : [ 0.75582 -0.08113]
iteracion | theta1  theta2 |  <H>
        1 |  1.186  2.010 | +0.8960
        2 |  1.026  2.030 | +0.7911
        3 |  0.802  2.069 | +0.5841
        4 |  0.493  2.143 | +0.1886
        5 |  0.086  2.276 | -0.5063
       10 | -1.519  3.109 | -2.9962
       20 | -1.571  3.142 | -3.0000
       30 | -1.571  3.142 | -3.0000
       40 | -1.571  3.142 | -3.0000
Energia final = -3.0000 (exacta: -3)
```

(El dibujo del circuito no se reproduce aquí.) Es determinista.

1.7. Interpretación

Las dos formas de calcular $\langle H \rangle$ dan -0.2308, y el gradiente por desplazamiento coincide con el numérico. Partiendo de $\theta = (1.3, 2.0)$ con $\langle H \rangle = +0.9484$, la energía baja a -0.51 en 5 iteraciones, a -2.996 en 10 y a -3.0000 en 20: el valor exacto del estado fundamental. Los parámetros terminan en $(-\pi/2,\ \pi)$.

1.8. Ejercicios

1. Cambia la tasa de aprendizaje a 0.1 y a 1.0. ¿Cuántas iteraciones hacen falta en cada caso?
2. Prueba otros parámetros iniciales: ¿converge siempre a -3?
3. Cambia el Hamiltoniano a $H = ZZ$ y observa qué energía mínima se obtiene.

1.9. Errores comunes

- Creer que VQE garantiza el mínimo global: el resultado depende del circuito de prueba y de los parámetros iniciales, y puede quedarse en un mínimo local.
- Olvidar que el valor esperado se estima con muchas mediciones en un equipo real; aquí se calcula exacto con el simulador.